



Instituto Superior de Engenharia de Lisboa

Licenciatura Engenharia Informática e de Computadores
Licenciatura Engenharia Informática e Multimédia

Ano letivo 2025/2026

Computação na Nuvem

Projeto Final

Docente Carlos Júnior

Turma 61N

Grupo 08

49454 Eduardo Tavares (LEIC)

50758 Leonor Medeiros (LEIM)

maio de 2026

Índice

1	Introdução	1
1.1	Arquitetura geral	1
2	Enquadramento teórico	2
2.1	Google Cloud Platform	2
2.2	Serviços utilizados	2
2.3	Conceitos e definições	3
2.3.1	gRPC e Protocol Buffers	3
2.3.2	Workers e padrão work-queue	3
3	Arquitetura	3
3.1	Acesso às bases de dados	4
3.2	Organização do código	4
3.3	Cloud hosting	5
4	Contratos de Comunicação	5
4.1	Contrato gRPC (Protobuf)	5
4.1.1	Operações funcionais (SF)	5
4.1.2	Operações de gestão de elasticidade (SG)	5
4.2	Mensagens Pub/Sub	6
4.3	Discussão das decisões	6
5	Fluxo de Dados e Interações	6
5.1	Descoberta e estabelecimento de ligação	6
5.2	Submissão e desacoplamento	7
5.3	Processamento assíncrono	7
5.4	Consulta de resultados	7
5.5	Considerações sobre o fluxo	8
6	Testes e operacionalidade	8
6.1	Validação do desacoplamento	8
6.2	Teste de elasticidade da Labels App	8
6.3	Teste de elasticidade dos servidores gRPC	9
6.4	Resumo dos resultados	9
7	Análise crítica	9
7.1	Pontos de falha	9
7.2	Aspetos de implementação	10
8	Conclusão	10

1 Introdução

Os objetivos deste projeto incluem aprender e praticar o planeamento e a realização de um sistema para submissão e execução de tarefas de computação, utilizando os serviços da Google Cloud Platform (GCP) para armazenamento, comunicação e computação, nomeadamente Cloud Storage, Firestore, Pub/Sub, Compute Engine e Cloud Run Functions.

Para isso, desenvolvemos um sistema, designado *CN2026Labels* (ou *Google Cloud Image Analyzer*), capaz de detetar características (*labels*), exemplo: *tree* ou *cat*, em ficheiros de imagem (JPG, PNG, etc.) e de traduzir essas características de inglês para português (árvore, gato). O sistema é elástico, aumentando ou diminuindo a capacidade de processamento através da variação do número de instâncias dos seus vários componentes.

ID do projeto GCP: cn2526-t4-g08

1.1 Arquitetura geral

A arquitetura do sistema utiliza diversos serviços do GCP. A Figura 2 mostra as diferentes interações entre os componentes do sistema.

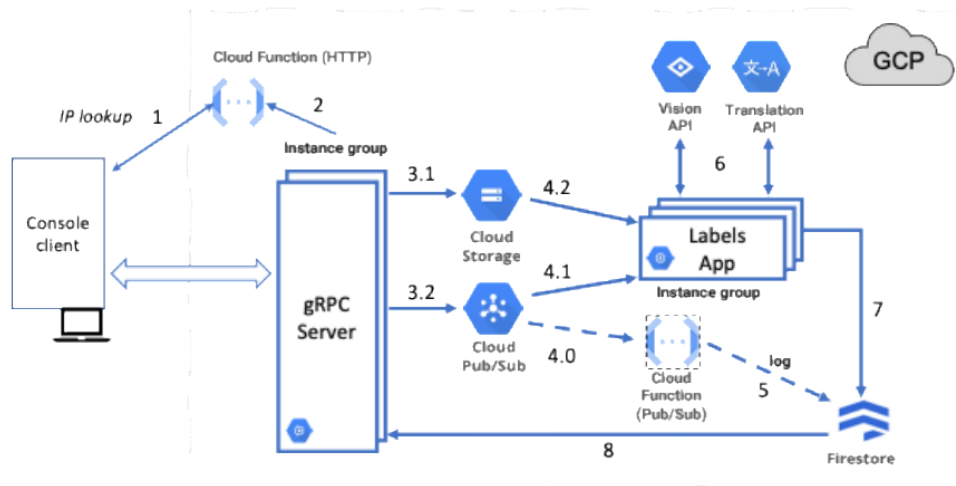


Figura 1: Componentes do CN2026Labels e principais interações funcionais.

Com a exceção da aplicação cliente, representada à esquerda na Figura 2, toda a infraestrutura está na nuvem do Google. Os principais componentes são:

- **Aplicação cliente** — aplicação na consola executada na máquina do utilizador, que comunica com o servidor gRPC através do seu IP externo;
- **Servidor gRPC** — recebe os pedidos do cliente, guarda as imagens no Cloud Storage e publica mensagens no Pub/Sub. Está implantado num *Managed Instance Group* para suportar elasticidade;
- **Labels App** — *worker* que consome mensagens do Pub/Sub, invoca a Vision API e a Translation API, e escreve os resultados no Firestore. Também é executado num *Managed Instance Group*;
- **Cloud Run Function (IP Lookup)** — função HTTP que devolve ao cliente a lista de IPs

externos das instâncias ativas do grupo de servidores gRPC;

- **Serviços** — Cloud Storage (armazenamento das imagens), Pub/Sub (tópico Imagens), Firestore (resultados), Vision API e Translation API.

2 Enquadramento teórico

2.1 Google Cloud Platform

A *Google Cloud Platform* (GCP) é a plataforma de *cloud computing* da Google, que disponibiliza um vasto conjunto de serviços geridos para armazenamento, computação, comunicação e processamento de dados. Cada serviço pode ser consumido de duas formas complementares: através da interface gráfica da consola web, útil para fazer a configuração inicial, inspeção de recursos e operações pontuais; ou através das respetivas APIs expostas pelos serviços, consumidas programaticamente. Neste trabalho, utilizámos a consola web para a configuração inicial e para verificar visualmente que o projeto estava a funcionar, e utilizámos as bibliotecas oficiais do Google para a linguagem Java.

2.2 Serviços utilizados

Utilizámos 5 serviços GCP. A descrição detalhada da forma como cada um é instanciado no projeto encontra-se na Secção 3.

Cloud Storage. É o serviço de armazenamento de objetos (*blobs*) organizados em *buckets*. É utilizado para armazenar as imagens submetidas pelos utilizadores, que serão posteriormente processadas pela Labels App.

Firestore. Base de dados NoSQL orientada a documentos, em modo nativo. No contexto deste projeto guarda a informação relevante sobre o processamento de cada ficheiro — nomeadamente o identificador do pedido, a data de processamento, as características detetadas nas imagens e as respetivas traduções.

Pub/Sub. Serviço de mensagens assíncrona baseado no modelo *publisher/subscriber*. Permite a troca desacoplada de mensagens entre os servidores gRPC (publicadores) e as aplicações de processamento de imagens (consumidores).

O Pub/Sub suporta dois modos de entrega de mensagens aos subscritores:

- **Push** — o Pub/Sub envia ativamente as mensagens para um *endpoint* HTTPS configurado no subscritor;
- **Pull** — os subscritores vão buscar (*pull*) as mensagens à subscrição quando estão disponíveis para as processar.

Como neste projeto os subscritores são aplicações Java a correr em VMs do Compute Engine, sem um *endpoint* HTTPS público pré-definido, foi utilizado o modo *pull*, que se adequa também ao padrão *work-queue* adotado.

Compute Engine. Serviço de *Infrastructure as a Service* (IaaS) que disponibiliza máquinas virtuais. Em particular, suporta *Managed Instance Groups* (MIGs) — grupos de VMs criadas a partir de um *Instance Template* comum, que podem ser redimensionados dinamicamente. Neste projeto, os MIGs alojam os servidores gRPC e as instâncias da Labels App.

Cloud Run Functions. Serviço *serverless* para execução de funções de curta duração, acionadas por eventos (*triggers*). No projeto é usado para implementar a função de *IP lookup* dos servidores gRPC (*trigger* HTTP) e, opcionalmente, uma função para registo (*logging*) de pedidos de processamento no Firestore (*trigger* Pub/Sub).

2.3 Conceitos e definições

2.3.1 gRPC e Protocol Buffers

O **gRPC** é a *framework* de *Remote Procedure Call* (RPC) *open-source* desenvolvida pela Google, baseada em HTTP/2 e Protocol Buffers. Permite que aplicações cliente e servidor comuniquem como se estivessem a invocar métodos locais, abstraindo a complexidade da rede.

Os **Protocol Buffers** (protobuf) constituem a *Interface Definition Language* (IDL) e o formato de serialização do gRPC. Um ficheiro `.proto` descreve as mensagens e os serviços, e o compilador `protoc` gera o código nas linguagens-alvo.

2.3.2 Workers e padrão work-queue

O conceito de *worker* designa uma unidade de execução — tipicamente uma *thread* ou um processo — que recebe tarefas a partir de uma fila partilhada e as processa de forma independente. Um exemplo clássico em Java são as *thread pools*, em que várias *threads* consomem tarefas de uma `BlockingQueue`, distribuindo naturalmente a carga entre si.

Neste projeto, o mesmo princípio é aplicado a uma escala distribuída: cada instância da Labels App funciona como um *worker* que consome mensagens de uma subscrição Pub/Sub partilhada. Esta arquitetura — **work-queue pattern** — apresenta várias propriedades úteis:

- **Distribuição automática de carga** — o próprio Pub/Sub distribui as mensagens pelos *workers* disponíveis, sem necessidade de um *load balancer* explícito;
- **Escalabilidade horizontal** — aumentar a capacidade de processamento resume-se a aumentar o número de *workers*;
- **Tolerância a falhas** — se um *worker* falhar antes de confirmar (*ack*) o processamento de uma mensagem, esta é reentregue a outro *worker*, garantindo a semântica *at-least-once*;
- **Desacoplamento** — o produtor (servidor gRPC) não precisa de saber quantos *workers* existem nem onde estão a correr.

Este padrão é particularmente adequado para tarefas independentes e relativamente demoradas (como é o caso do processamento de uma imagem com chamadas à Vision API e Translation API), em que a latência individual de cada tarefa pode ser superior ao *overhead* de a colocar em fila.

3 Arquitetura

A estrutura decidida para o trabalho contém três aplicações (aplicação cliente, servidor, e um worker que processa a imagem recebida pelo servidor), duas bases de dados, uma tipo bucket para armazenamento de imagens, e firestore para dados relacionados às imagens, e os protocolos de comunicação entre cliente-servidor (grpc) e servidor-worker (pub/sub).

A comunicação gRPC entre o cliente e servidor é bidirecional, já que ambos necessitam de enviar informação um ao outro. No caso do pub/sub, o servidor é o único que envia informação, devido

ao propósito da comunicação ser informar os workers Labels App que têm de processar a imagem indicada pelo servidor.

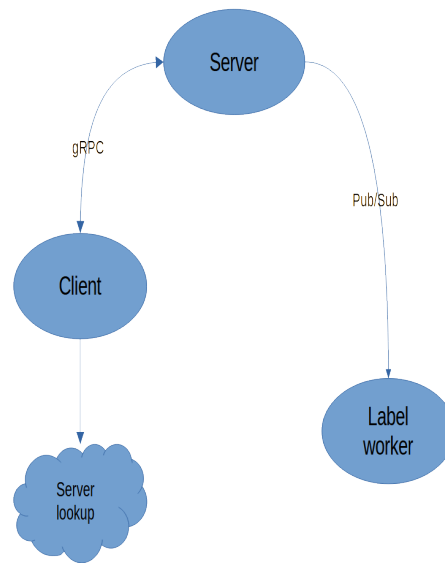


Figura 2: Arquitetura

Além disso, também podemos ver na imagem cliente a fazer uso da função server lookup. esta monitoriza os servidores, e indica os fornece os Ips ao cliente, mudando a conexão de ser definida de compilação para runtime, incluindo mudar de instância de servidor durante a execução do cliente se o servidor escolhido ficar indisponível.

3.1 Acesso às bases de dados

Na utilização do Firestore, cabe ao labels App escrever a informação necessária sobre as imagens, enquanto o servidor está encarregado ler a informação para a delegar ao cliente.

Enquanto se pode assumir que o mesmo acontece com o Cloud Storage, onde o servidor escreve as imagens e o labels App lê para as analisar, anotamos que a leitura não acontece, e utilizamos o serviço Google que permite detetar os labels remotamente.

3.2 Organização do código

Cada aplicação e cloud function tem a sua pasta pessoal para definir e implementar a sua lógica, tendo em conta que devem funcionar independentemente. O gRPC precisa de definir o seu contrato e para isso criamos um projeto maven extra na sua pasta. Em oposição ao pubsub em que enviamos texto no formato preferido, neste caso escolhemos utilizar o formato JSON. Com os plugins da Google, compilamos e instalamos um jar, que disponibiliza logo os métodos de comunicação para o cliente e a interface de implementação de cada método no contrato para o servidor.

Havia a escolha de criar projetos extra, os que podiam incluir aspetos comuns, como a classe representativa do JSON utilizado no pub/sub e as strings identificadoras das coleções da firestore utilizadas. Nenhum destes projetos extra foi utilizado.

3.3 Cloud hosting

Depois de criar os ficheiros JAR necessários através do Maven, pode-se utilizar a Cloud para fazer hosting do código. No caso dos servidores e da labels app, começamos por criar máquinas virtuais para cada um, e utilizamos o bitvise para ambos instalar Java 25 e fazer upload do JAR em conjunto com a sua service account key. A partir desta máquina criamos uma imagem, seguida de um template, a qual contém um ficheiro startup para começar o código java.

Esse template, permite criar uma Managed Instance Group, um grupo de máquinas virtuais que o Google Cloud gere por si própria. Esta estrutura dada pela Cloud permite elasticidade nos recursos físicos utilizados a cada momento, indicando na definição destes grupos que se deve criar mais instâncias das máquinas virtuais ao notar uma subida na carga imposta na aplicação.

4 Contratos de Comunicação

Num sistema distribuído como este, em que diferentes componentes (cliente, servidor gRPC, Labels App, Firestore, Pub/Sub) são desenvolvidos e implantados de forma independente, é essencial definir contratos de comunicação. Estes contratos garantem a interoperabilidade e desacoplamento entre as partes.

Identificámos três contratos formais que suportam o sistema:

- **Contrato gRPC (Protobuf):** comunicação entre cliente e servidor gRPC;
- **Formato das mensagens Pub/Sub:** comunicação assíncrona entre o servidor gRPC (produtor) e a Labels App (consumidor);
- **Estrutura dos documentos no Firestore:** contrato de dados partilhado entre a Labels App (escritor) e o servidor gRPC (leitor).

.

4.1 Contrato gRPC (Protobuf)

Com os conceitos já revistos anteriormente 2.3.1, nesta secção explicamos brevemente as operações definidas.

4.1.1 Operações funcionais (SF)

As operações funcionais são as que estão diretamente expostas ao utilizador final:

- **Submissão de imagem:** recebe o conteúdo do ficheiro em *stream* de blocos e devolve um identificador único de pedido.
- **Consulta de resultados:** a partir do identificador devolvido na submissão, devolve a lista de *labels* detetadas, as respetivas traduções e a data de processamento.
- **Pesquisa por característica e intervalo de datas:** devolve os nomes dos ficheiros armazenados entre duas datas que contêm uma determinada *label*.

4.1.2 Operações de gestão de elasticidade (SG)

A elasticidade do projeto baseia-se na capacidade de aumentar ou diminuir o número de instâncias ativas nos Grupos de Instâncias (*Managed Instance Groups*), adaptando assim a capacidade de pro-

cessamento à carga de pedidos. É possível redimensionamento do grupo de instâncias dos servidores gRPC; e redimensionamento do grupo de instâncias da Labels App.

4.2 Mensagens Pub/Sub

Este protocolo utiliza o padrão *work-queue* explicado na seção 2.3.2, permitindo a distribuição de trabalho. Neste ambiente específico, o consumidor utiliza o mecanismo *acknowledge* para indicar sucesso. Se isto não acontecer, volta para a pool de tarefas.

Cada mensagem pub/sub tem em formato json necessário para encontrar a imagem guardada nos buckets do projeto.

4.3 Discussão das decisões

Transparência e desacoplamento. O uso de contratos baseados em IDLs (Protobuf no gRPC, esquema acordado no Pub/Sub, estrutura de documento no Firestore) permite que cada componente evolua de forma independente, desde que o contrato seja respeitado. Por exemplo, é possível reescrever a Labels App noutra linguagem sem alterar o servidor gRPC.

Eficiência e latência. A escolha de gRPC sobre HTTP/2, em detrimento de uma API REST/JSON, justifica-se pela serialização binária mais compacta, pela multiplexagem de chamadas sobre a mesma ligação e pelo suporte nativo a *streaming*, que foi essencial para o upload de imagens.

Fiabilidade. O mecanismo de *ack* do Pub/Sub garante que as imagens são processadas mesmo em caso de falha de um *worker*, uma vez que uma mensagem não confirmada é reentregue. Esta propriedade é fundamental para que o sistema seja resiliente sem necessidade de lógica adicional de retry na aplicação.

5 Fluxo de Dados e Interações

Esta seção descreve a sequência lógica de ações realizadas pelo sistema, desde o pedido inicial do cliente até à obtenção do resultado final, correlacionando cada passo com os serviços GCP envolvidos. A numeração utilizada acompanha a sequência de interações representada na Figura 2.

5.1 Descoberta e estabelecimento de ligação

Nesta fase inicial o sistema resolve dinamicamente a localização dos servidores gRPC, permitindo que o número de instâncias possa variar sem que o cliente tenha de conhecer previamente os seus endereços.

A aplicação cliente começa por invocar a **Lookup Function** — uma Cloud Run Function com *trigger* HTTP — que consulta o *Managed Instance Group* dos servidores gRPC e devolve a lista de endereços IP externos das instâncias ativas. O cliente escolhe um dos IPs da lista e tenta estabelecer a ligação gRPC; se a ligação falhar (por exemplo, porque a instância acabou de ser removida do grupo), tenta um IP alternativo ou repete o *lookup* para obter uma lista atualizada.

Esta indireção é fundamental para a elasticidade do sistema: o cliente nunca tem um endereço *hardcoded* e adapta-se automaticamente a alterações no número de servidores.

5.2 Submissão e desacoplamento

A segunda fase corresponde à entrada da imagem no sistema e à passagem do controlo para a parte assíncrona.

Após a ligação estar estabelecida, o utilizador seleciona no menu do cliente a opção de submeter uma imagem e fornece o *path* local do ficheiro. O cliente envia o conteúdo da imagem ao servidor gRPC em *client-side streaming*, dividido em blocos de bytes. À medida que o servidor recebe os blocos, vai escrevendo o ficheiro no Cloud Storage como um *blob* num *bucket* pré-configurado, ao qual é atribuído um identificador único.

Concluído o *upload*, o servidor gRPC publica uma mensagem num tópico do Pub/Sub contendo três campos: o identificador do pedido, o nome do *bucket* e o nome do *blob*. Imediatamente a seguir, devolve esse mesmo identificador ao cliente, que pode então usá-lo para consultar o resultado quando este estiver disponível.

Logging opcional. Opcionalmente, uma Cloud Function com *trigger* Pub/Sub recebe a mesma mensagem e regista o pedido numa coleção dedicada do Firestore, permitindo um registo independente de auditoria, sem interferir com o processamento principal.

5.3 Processamento assíncrono

As instâncias da Labels App, em execução no respetivo *Managed Instance Group*, partilham uma única subscrição *pull* do tópico Pub/Sub. Seguindo o padrão *work-queue*, cada mensagem é entregue a apenas um *worker*, distribuindo automaticamente a carga sem necessidade de um *load balancer* adicional.

Ao receber uma mensagem, o *worker* executa a seguinte sequência:

1. Lê a imagem do Cloud Storage usando o nome do *bucket* e do *blob* indicados na mensagem;
2. Invoca a **Vision API**, que devolve uma lista de *labels* detetadas, cada uma acompanhada de uma pontuação de confiança (entre 0 e 1) e de um valor de *topicality*;
3. Invoca a **Translation API** para traduzir cada *label* de inglês para português;
4. Escreve no **Firestore** um documento identificado pelo ID do pedido, contendo as *labels* traduzidas, as respetivas pontuações e a data de processamento;
5. Confirma (*ack*) a mensagem no Pub/Sub, removendo-a definitivamente da subscrição.

5.4 Consulta de resultados

A última fase fecha o ciclo de vida do pedido.

A qualquer momento após a submissão, o utilizador pode selecionar a opção de consulta no menu do cliente e fornecer o identificador devolvido na submissão. O cliente envia esse ID ao servidor gRPC, que consulta o documento correspondente no Firestore e devolve as *labels* traduzidas, as pontuações e a data de processamento. O cliente apresenta então esta informação ao utilizador.

As restantes operações de consulta funcionam de forma semelhante: pesquisa de imagens por intervalo de datas e *label*, e *download* de uma imagem a partir do seu ID. Em ambos os casos o servidor gRPC atua como intermediário entre o cliente e os serviços GCP (Firestore para a pesquisa, Cloud Storage para o *download*).

5.5 Considerações sobre o fluxo

Justificação do assincronismo. A introdução do Pub/Sub entre o servidor gRPC e a Labels App desacopla a submissão do processamento. Esta separação tem várias consequências positivas: o cliente recebe rapidamente o identificador do pedido e não fica bloqueado enquanto a imagem é analisada (operação que pode demorar devido às chamadas à Vision API e à Translation API); picos temporários de submissões são absorvidos pela fila, sem causar falhas; e o servidor gRPC mantém-se leve, dedicando-se apenas à I/O do *upload* e à publicação da mensagem.

Suporte à elasticidade. Cada uma das fases é independente em termos de escala. Em particular, o fluxo permite aumentar o número de instâncias da Labels App para responder a um pico de submissões sem afetar a disponibilidade do servidor gRPC, e vice-versa. As operações de gestão (SG) expostas pela API gRPC permitem controlar explicitamente o tamanho de cada um dos *Managed Instance Groups*.

Idempotência e duplicação. Como o Pub/Sub garante entrega *at-least-once*, é possível (embora raro) que a mesma mensagem seja entregue mais do que uma vez — por exemplo, se um *worker* demorar a confirmar o *ack*. No estado atual, o sistema assume que a reescrita do documento Firestore com o mesmo ID é inofensiva (idempotente por sobreposição).

6 Testes e operacionalidade

Esta secção descreve a sequência de testes realizada para validar a operacionalidade do sistema, em particular o desacoplamento entre componentes e o comportamento elástico dos *Managed Instance Groups*. A sequência seguida corresponde ao cenário de demonstração obrigatório.

6.1 Validação do desacoplamento

Com a Labels App ainda em zero instâncias, foram submetidas várias imagens através da aplicação cliente. Confirmou-se que:

- os *blobs* foram corretamente armazenados no Cloud Storage e o servidor gRPC devolveu os respetivos identificadores ao cliente;
- as mensagens correspondentes foram publicadas no tópico Pub/Sub e ficaram **retidas na subscrição** (visível na consola GCP através da métrica *Unacked messages*);
- nenhum documento foi criado no Firestore, o que confirma que o servidor gRPC não escreve resultados nem depende dos *workers* para concluir a operação de submissão.

Este passo demonstra na prática o desacoplamento entre produtor e consumidor: a indisponibilidade total dos *workers* não impede o sistema de aceitar novos pedidos.

6.2 Teste de elasticidade da Labels App

Em seguida, o grupo da Labels App foi redimensionado para **duas** instâncias através da operação SG correspondente. Após o tempo de arranque das VMs e da JVM, observou-se que:

- as mensagens acumuladas na subscrição começaram a ser consumidas, com a métrica *Unacked messages* a decrescer até zero;

- os documentos foram aparecendo no Firestore, identificados pelos IDs devolvidos anteriormente ao cliente;
- as consultas subsequentes pelo cliente, usando esses IDs, devolveram corretamente as *labels* traduzidas e a data de processamento.

O facto de o trabalho acumulado ter sido processado em paralelo pelas duas instâncias, sem duplicação de documentos, valida o padrão *work-queue* adotado.

6.3 Teste de elasticidade dos servidores gRPC

Por fim, o grupo dos servidores gRPC foi também redimensionado para **duas** instâncias. A aplicação cliente foi reiniciada e invocou novamente a *Lookup Function*, que devolveu agora dois IPs externos em vez de um. Foi escolhido o novo servidor e repetida uma submissão, confirmando que ambas as instâncias gRPC operam sobre o mesmo conjunto de recursos partilhados (Cloud Storage, Pub/Sub, Firestore) sem necessidade de qualquer sincronização adicional entre si.

6.4 Resumo dos resultados

Os testes realizados confirmam que o sistema cumpre os requisitos funcionais (submissão por *streaming*, processamento de imagens, consulta de resultados) e os requisitos de elasticidade (controlo explícito do número de instâncias de cada componente). Não foram observados erros ou falhas durante a sequência descrita.

7 Análise crítica

7.1 Pontos de falha

Sendo um sistema distribuído na *cloud*, herda as fragilidades intrínsecas deste tipo de arquiteturas.

Fiabilidade e latência da rede. A primeira falácia — “a rede é fiável” — é particularmente relevante para a aplicação cliente, que executa fora da *cloud*: uma falha na ligação à Internet inviabiliza tanto a submissão como a consulta. Internamente, embora as comunicações entre serviços GCP usem a *backbone* da Google e sejam altamente fiáveis, continuam sujeitas a falhas pontuais. A escolha de uma região europeia (**europe-west1**) para todos os recursos do projeto minimiza ainda a latência inter-serviço; usar, por exemplo, um *bucket* numa região asiática degradaria significativamente o *throughput* sem qualquer benefício funcional.

Teorema CAP e consistência. O Teorema CAP estabelece que, perante uma partição de rede, um sistema distribuído tem de escolher entre Consistência e Disponibilidade. Os serviços usados neste projeto fazem essa escolha de formas diferentes: o Firestore em modo nativo oferece consistência forte para leituras de um único documento, o que é adequado ao caso de uso (consulta pelo ID do pedido); por outro lado, a propagação de alterações ao IAM ou às regras de *firewall* é *eventually consistent*, podendo demorar alguns segundos a refletir-se em todos os *endpoints*.

Indisponibilidade de serviços. Apesar dos SLAs muito elevados anunciados pela Google (tipicamente *four to five nines*), nenhum serviço está imune a falhas parciais. No estado atual, o sistema é resiliente a falhas dos *workers* (graças ao mecanismo *ack* do Pub/Sub), mas não trata explicitamente erros transitórios da Vision API ou da Translation API. Uma melhoria natural seria adicionar *retry* com *exponential backoff* a essas chamadas.

7.2 Aspectos de implementação

Ambiente de execução. Todo o código foi desenvolvido em Java sobre o JDK 25, gerido pelo sistema de *build* Maven. O Maven é particularmente adequado a este projeto porque permite declarar de forma centralizada as várias bibliotecas oficiais da Google (`google-cloud-storage`, `google-cloud-firestore`, `google-cloud-pubsub`, `google-cloud-vision`, `google-cloud-translate`, `grpc-java`), bem como invocar o *plugin* de Protocol Buffers para gerar automaticamente as *stubs* a partir do ficheiro `.proto`.

Comunicação gRPC. O uso de Protocol Buffers como *língua franca* binária entre cliente e servidor traz duas vantagens face a uma API REST sobre HTTP/1.1: mensagens significativamente mais compactas (pelo menos em comparação para o conteúdo binário das imagens) e a possibilidade de usar *streaming*, usado para a submissão de ficheiros. As *stubs* geradas oferecem variantes bloqueantes e não bloqueantes; nas operações unárias do cliente foi usada a variante bloqueante por simplicidade, enquanto no *upload* a variante assíncrona é intrínseca à utilização de *streaming*.

Persistência no Firestore. Os documentos guardados foram mantidos propositadamente pequenos — bem abaixo do limite de 1 MiB por documento — contendo apenas os campos estritamente necessários (ID, *labels*, pontuações, data). Para suportar a operação de pesquisa por intervalo de datas e *label*, foi necessário criar um **índice composto** na consola GCP, uma vez que o Firestore não cria automaticamente índices para consultas que combinam um filtro de igualdade (`labelNames array-contains`) com um filtro de intervalo (`processingDate`).

8 Conclusão

O desenvolvimento permitiu implementar um sistema distribuído na Cloud com suporte a elasticidade controlada, integrando múltiplos serviços da Google Cloud Platform.

Os objetivos principais foram atingidos:

- submissão de imagens por *streaming*;
- processamento assíncrono através de Pub/Sub;
- deteção e tradução de *labels*;
- armazenamento e consulta de resultados;
- controlo programático da elasticidade dos componentes.

A utilização de Pub/Sub revelou-se central para o desacoplamento entre componentes e para a distribuição de carga pelos *workers*. O uso de gRPC e Protocol Buffers simplificou a comunicação entre cliente e servidor e permitiu suporte eficiente a *streaming*.

O projeto permitiu consolidar conhecimentos práticos relacionados com computação distribuída, serviços cloud, comunicação RPC e gestão de infraestrutura na Google Cloud Platform.